



Proyecto Final

Diseño de IoT y Sistemas Embebidos

Integrantes del grupo

Agustina Barcigalupe

Pierina Borsieri

Sofia Nicoletti

Adrián Tesore

Docentes del curso

Nicolas Calarco

Pablo Alonso

4 de julio de 2025

Marco Teórico.....	2
ESP32-S2 LyraT-8311A v1.2	2
Introducción.....	2
Arquitectura de Hardware	2
Sistema de Archivos SPIFFS en ESP32-S2	3
Introducción.....	3
Características Principales	3
Arquitectura de SPIFFS.....	3
Detalle de las especificaciones.....	4
Reproducción.....	4
Características Principales	4
Configuración de Hardware	4
Formato de los Audios	5
Diagrama de secuencia:	7
Conectividad y configuración	8
Introducción.....	8
Componente wifi_connection	8
Componente web_server	11
Logger	13
MQTT & Broker MQTT.....	15
HEARTBEAT LED	15
TOUCH PAD	16
NTP	17

Marco Teórico

ESP32-S2 LyraT-8311A v1.2

Introducción

La ESP32-S2 LyraT-8311A v1.2 es una placa de desarrollo diseñada por Espressif Systems para aplicaciones de audio embebido. Está optimizada para trabajar con el microcontrolador ESP32-S2, ofreciendo capacidades avanzadas de reproducción y grabación de audio, lo que la hace ideal para proyectos de interfaces de voz, asistentes virtuales y sistemas de control por comandos de audio.

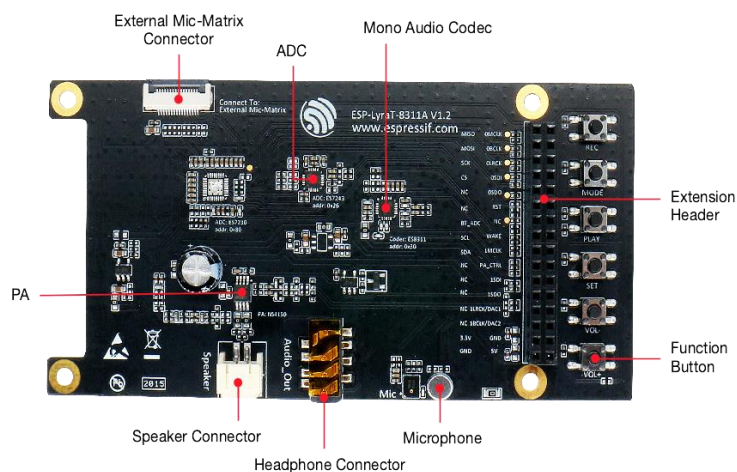
Características Principales

- Codec ES8311: Conversión de señales analógicas a digitales (ADC) y digitales a analógicas (DAC) con soporte para control de volumen, ganancia y configuración vía I2C.
- Amplificador de Potencia: Salida de hasta 3W para altavoces externos, controlado mediante GPIO.
- Conectividad de Audio: Incluye conectores para auriculares, altavoces y micrófonos, con soporte para micrófonos electret y MEMS.
- Interfaz I2S: Comunicación serial para transmisión de datos de audio digital entre el ESP32-S2 y el codec ES8311.
- Botones Programables: Seis botones táctiles configurables para control directo de funciones de audio.

Arquitectura de Hardware

Interfaz de Comunicación

- I2S: Protocolo para transmisión de audio digital, utilizado para conectar el ESP32-S2 con el codec ES8311.
- I2C: Protocolo para configuración y control del codec ES8311.



Sistema de Archivos SPIFFS en ESP32-S2

Introducción

El SPIFFS (SPI Flash File System) es un sistema de archivos ligero diseñado específicamente para dispositivos embebidos que utilizan memoria flash. En el caso del ESP32-S2, SPIFFS permite almacenar y gestionar archivos en una partición dedicada de la memoria flash, proporcionando una solución eficiente para el almacenamiento persistente de datos.

Características Principales

1. Diseño Ligero: Optimizado para dispositivos con recursos limitados.
2. Wear Leveling: Distribuye el desgaste de la memoria flash para prolongar su vida útil.
3. Almacenamiento Persistente: Los datos permanecen almacenados incluso después de reiniciar el dispositivo.
4. Flexibilidad: Permite almacenar archivos de tamaño variable, a diferencia de NVS (Non-Volatile Storage), que está diseñado para pares clave-valor.
5. Compatibilidad: Integrado en el framework ESP-IDF, lo que facilita su uso en proyectos basados en ESP32-S2.

Arquitectura de SPIFFS

SPIFFS utiliza una partición dedicada en la memoria flash, definida en el partition table del proyecto. En este caso, se asignaron 320 KB para SPIFFS:

Detalle de las especificaciones

Reproducción

El componente **audio_controller** es un componente de control de audio desarrollado específicamente para la placa ESP32-S2 LyraT 8311v1.2. Implementa una arquitectura basada en colas de eventos asíncronos utilizando FreeRTOS/queues, proporcionando un control robusto y thread-safe de la reproducción de audio mediante el codec ES8311 con interfaz I2S.

Características Principales

- Reproducción en Loop Infinito: Las pistas se reproducen continuamente hasta recibir otro comando
- Control Remoto Multicapa: Compatible con MQTT, Web Server y Touch Pad
- Control de Volumen Digital: Incrementos/decrementos de 10 en 10 (rango 0-100)
- Sistema de Auditoría: Integración completa con sistema de logger para registro de eventos
- Configuración Flexible: Sample rate, volumen inicial y micrófono configurables
- Thread-Safe: Arquitectura robusta con mutex y colas de eventos FreeRTOS
- Máquina de estado simple con Cambio automático de pistas conservando estado de reproducción

Configuración de Hardware

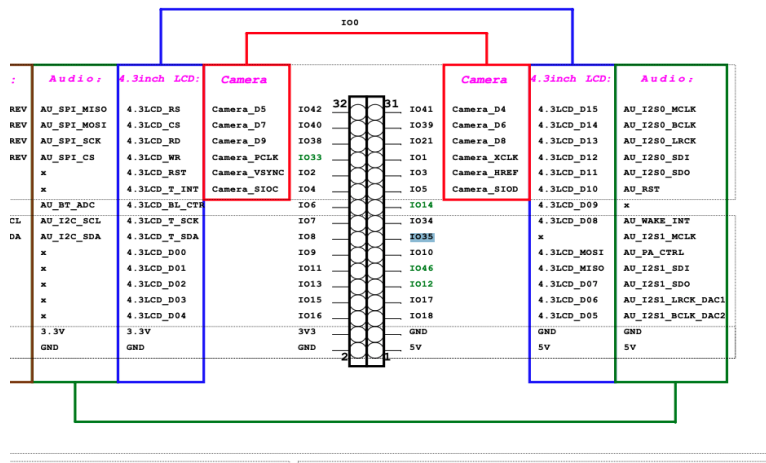
Especificaciones de la Placa de audio LyraT 8311 v1.2

Pines I2S:

- **I2S_NUM**: 1 (I2S1 según esquemático)
- **I2S_MCK_IO**: GPIO_NUM_35 (Master Clock)
- **I2S_BCK_IO**: GPIO_NUM_18 (Bit Clock)
- **I2S_WS_IO**: GPIO_NUM_17 (Word Select)
- **I2S_DO_IO**: GPIO_NUM_12 (Speaker Output)
- **I2S_DI_IO**: GPIO_NUM_46 (Microphone Input)

Pines I2C:

- **I2C_NUM**: 0 (I2C0)
- **I2C_SCL_IO**: GPIO_NUM_7 (Clock)
- **I2C_SDA_IO**: GPIO_NUM_8 (Data)



Pin Configuration for Subboard

Formato de los Audios

Los audios utilizados en el proyecto están embebidos en la memoria flash del ESP32-S2 en formato PCM (Pulse Code Modulation).

Este formato es:

- Resolución: 16 bits por muestra
- Frecuencia de Muestreo: 8 kHz (8000 muestras por segundo)
- Canales: Estéreo (2 canales)
- Codificación: Sin compresión, lo que permite una reproducción directa sin necesidad de decodificación.

Los archivos PCM están definidos como arreglos de bytes en la memoria flash y se referencian mediante etiquetas de ensamblador (asm) en el código.

```
// External references to embedded audio data
extern const uint8_t music_pcm_start[] asm("_binary_victory8bit_pcm_start");
extern const uint8_t music_pcm_end[] asm("_binary_victory8bit_pcm_end");
```

Esto permite que los datos de audio sean accesibles directamente desde el programa sin necesidad de cargar archivos desde un sistema de almacenamiento externo.

Máquina de Estados

El controlador de audio utiliza una máquina de estados simple para gestionar la reproducción:

- STOPPED: Estado inicial, sin reproducción activa.
- PLAYING: Reproducción activa en loop continuo.
- PAUSED: Reproducción detenida, posición de la pista mantenida.
- NEXT/PREVIOUS: Cambio de pista, manteniendo el estado de reproducción si estaba activo.

Auditoría y Logging

Cada evento de reproducción, pausa, cambio de pista o ajuste de volumen se registra automáticamente en el sistema de logs. Esto permite realizar un seguimiento detallado de las acciones realizadas por el usuario, ya sea mediante MQTT, Web Server o Touch Pad.

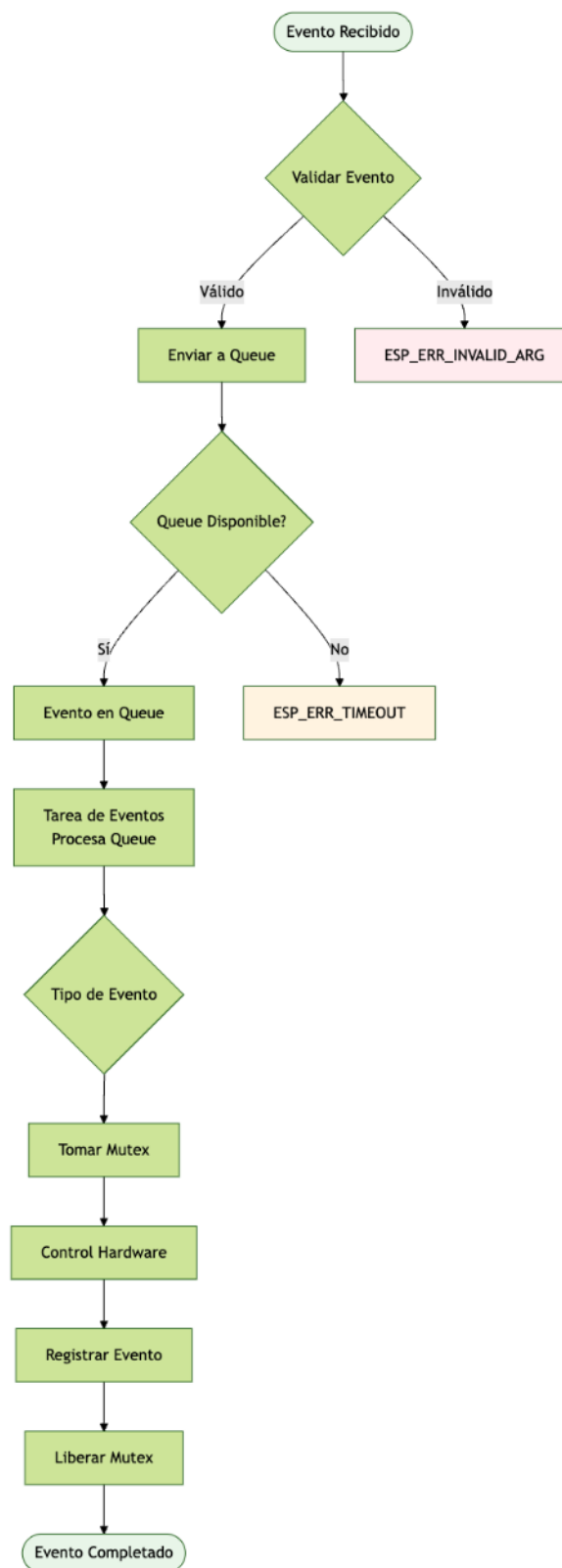
Configuración Flexible

El controlador permite configurar:

- Frecuencia de Muestreo: Por defecto 8 kHz, pero puede ajustarse según las necesidades del proyecto.
- Volumen Inicial: Configurable al iniciar el sistema.
- Micrófono: Opcional, habilitado/deshabilitado según la configuración.
- Integración con FreeRTOS

El sistema utiliza colas de eventos (xQueue) para procesar los comandos de control de forma asíncrona. Esto asegura que las operaciones sean thread-safe y no bloqueen el flujo principal del programa. Además, se utilizan mutex para proteger el acceso concurrente al hardware.

Diagrama de secuencia:



Conectividad y configuración

Introducción

La conectividad del sistema se gestiona mediante los componentes **wifi_connection** y **web_server**, que trabajan en conjunto para proporcionar una experiencia de configuración y control intuitiva. Estos módulos permiten que la ESP32-S2 opere en dos modos principales: **Access Point (AP)** y **Estación (STA)**, dependiendo de si existe o no una configuración previa almacenada en la memoria flash (NVS). Además, el sistema incluye una interfaz web para configurar parámetros de red y controlar la reproducción de audio.

Componente **wifi_connection**

El componente **wifi_connection** gestiona la conexión WiFi de la ESP32-S2, permitiendo que el dispositivo opere en modo AP para configuración inicial o en modo STA para conectarse automáticamente a una red wifi previamente configurada.

Funcionamiento

1. Inicio del Sistema:
 - a. El componente intenta cargar la configuración de **WiFi** y **MQTT** desde la **NVS**.
 - b. Si no encuentra datos válidos, activa el modo AP para que el usuario configure la red.
 - c. Si encuentra datos válidos, configura el modo STA y se conecta automáticamente a la red WiFi.
2. Modo Access Point (AP):
 - a. SSID: El nombre de la red incluye el identificador del chip **ESP32-Config**.
 - b. Contraseña: **config123**.
 - c. IP: Dirección estática (**192.168.4.1**).
 - d. Propósito: Permitir al usuario configurar la red **WiFi** y el broker **MQTT**.

Dentro de lo que son las opciones web podremos ingresar a:



Para poder configurar la red wifi y el broker y topic mqtt.

 **Configuración WiFi**

Red WiFi (SSID):

Contraseña WiFi:

 **Configuración MQTT**

Broker MQTT:

Tópico MQTT:

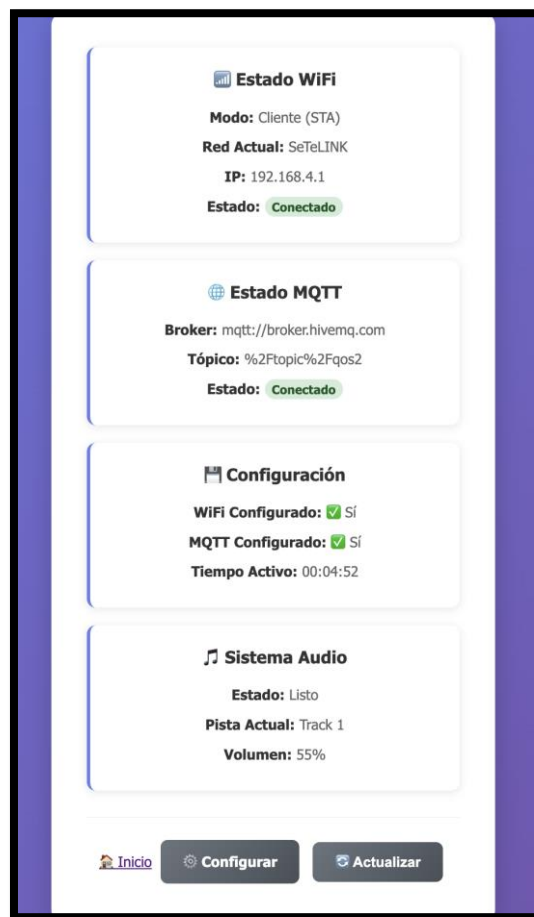
 Guardar Configuración

 Ver Estado

 Borrar Configuración

Una vez hayamos guardado la configuración de la red wifi, se procede al guardado en la nvs de la config y luego se reiniciará el dispositivo dando 10 segundos de espera para que pase esto. Una vez reiniciado se levantará del nvs estas configs y se setearan en el wifi y el mqtt.

En el estado de la esp32-s2 podremos ver las distintas configuraciones preestablecidas.



También el webserver nos da la posibilidad de reiniciar o volver a un estado “de fábrica”



que básicamente lo que hace esto es eliminar todas las configuraciones que guardamos previamente en la nvs.

3. Modo Estación (STA):
 - a. Conexión Automática: Se conecta automáticamente a la red configurada.
 - b. Reconexión: Intenta reconectar hasta 5 veces en caso de desconexión.
 - c. MQTT: Establece comunicación con el broker **MQTT** y comienza a escuchar comandos.

Configuración WiFi y MQTT

- Parámetros Configurables:
 - WiFi: SSID y contraseña.
 - **MQTT**: URI del broker y topic.
- Almacenamiento en NVS:
 - Los parámetros se guardan en claves específicas (**wifi_ssid**, **wifi_pass**, **mqtt_uri**, **mqtt_topic**).
 - Se utiliza un flag (**wifi_configured**) para indicar si la configuración está completa.

Eventos Manejados

- **WIFI_EVENT_STA_DISCONNECTED**: Gestiona desconexiones y reconexiones.
- **IP_EVENT_STA_GOT_IP**: Maneja la obtención de direcciones IPv4.
- **IP_EVENT_GOT_IP6**: Maneja la obtención de direcciones IPv6.

Componente web_server

El componente **web_server** implementa un servidor web que permite al usuario configurar parámetros del sistema y controlar la reproducción de audio desde un navegador.

Funcionamiento

1. Interfaz Web:
 - a. Sirve páginas HTML embebidas para configuración y control.
 - b. Permite al usuario interactuar con el sistema desde cualquier dispositivo conectado a la red WiFi creada por el modo AP.
2. Configuración del Sistema:
 - a. Página **/config**: Permite ingresar SSID, contraseña, URI del broker MQTT y topic.
 - b. Página **/save_config**: Procesa los datos enviados y los guarda en la NVS.
 - c. Página **/clear_config**: Borra configuraciones y reinicia el sistema.
3. Control de Audio:
 - a. Página **/**: Incluye botones para enviar comandos como PLAY, PAUSE, NEXT, PREVIOUS, VOLUME_UP, y VOLUME_DOWN.
 - b. Los comandos se publican vía MQTT en el topic configurado.
4. Estado del Sistema:
 - a. Página **/status**: Muestra información actualizada sobre la conexión WiFi, MQTT, y tiempo activo del sistema.

Manejo de Solicitudes HTTP

Endpoints:

- **/**: Página principal con el modo actual del sistema.
- **/config**: Página de configuración.
- **/save_config**: Procesa y guarda configuraciones.
- **/status**: Muestra el estado del sistema.
- **/clear_config**: Borra configuraciones y reinicia el sistema.
- **/style.css**: Hoja de estilos.

Logger

El componente logger es responsable de registrar eventos relacionados con la reproducción de audio, como **PLAY**, **PAUSE**, **NEXT**, **PREVIOUS**, **STOP**, y ajustes de volumen (**VOLUME_UP** y **VOLUME_DOWN**). Los eventos se almacenan en un buffer circular de tamaño fijo (20 eventos) y se guardan de forma persistente en el sistema de archivos SPIFFS.

Estructura de los Logs

Cada evento registrado incluye:

- Tipo de Evento: Identifica la acción realizada (ejemplo: **PLAY**, **PAUSE**).
- Timestamp: Hora exacta del evento, sincronizada con el tiempo actual mediante el componente NTP.
- Número de Secuencia: Un contador global que asegura el orden cronológico de los eventos.

La estructura de un evento es la siguiente:

```
typedef struct {
    logger_event_type_t type;    // Tipo de evento
    time_t timestamp;           // Timestamp en segundos
    uint32_t sequence_number;    // Número de secuencia global
} logger_event_t;
```

Almacenamiento Persistente

Los eventos se guardan en el sistema de archivos SPIFFS en la partición dedicada definida en el partition table. Esto permite que los eventos persistan incluso después de reiniciar el dispositivo.

Proceso de Guardado

1. Buffer Circular: Los eventos se almacenan en un buffer circular en RAM. Cuando el buffer está lleno, los eventos más antiguos son sobrescritos (FIFO).
2. Guardado Asíncrono: Una tarea de baja prioridad en FreeRTOS guarda los eventos en SPIFFS de forma no bloqueante.
3. Auto-Guardado: El sistema guarda automáticamente los eventos al apagar el dispositivo mediante un handler de apagado registrado con `esp_register_shutdown_handler`.

Sincronización de Tiempo

El timestamp de cada evento se obtiene utilizando el componente NTP. Antes de inicializar el logger, el sistema sincroniza el tiempo con un servidor NTP para garantizar que los registros tengan una marca de tiempo precisa.

Envío de Logs por MQTT

Cuando el dispositivo se conecta al broker MQTT, los logs almacenados en el buffer circular se envían en formato JSON. Este proceso ocurre en la tarea **sync_events_task** del main.c.

Proceso de Envío

- Recuperación de Logs: Los eventos se obtienen del buffer circular en orden cronológico.
- Conversión a JSON: Cada evento se convierte a un objeto JSON utilizando la función `logger_event_to_json`.
- Publicación MQTT: Los eventos se concatenan en un array JSON y se envían al broker MQTT en el topic configurado (**/esp32/audio/events**).

Ejemplo de JSON generado:

```
[
  {
    "type": "PLAY",
    "sequence": 1,
    "timestamp": "2025-07-04 12:00:00",
    "timestamp_raw": 1688472000,
    "date": "2025-07-04 12:00:00"
  },
  {
    "type": "PAUSE",
    "sequence": 2,
    "timestamp": "2025-07-04 12:05:00",
    "timestamp_raw": 1688472300,
    "date": "2025-07-04 12:05:00"
  }
]
```

Partition Table

El partition table define las particiones de memoria flash utilizadas por el sistema. En este proyecto, se configuró una partición dedicada para SPIFFS:

```
# ESP-IDF Partition Table para Flash 2MB
# Name, Type, SubType, Offset, Size, Flags
nvs,      data, nvs,      0x9000, 0x6000,
phy_init, data, phy,      0xf000, 0x1000,
factory,  app,  factory, 0x10000, 0x180000,
spiffs,   data, spiffs,  ,      0x50000,
```

MQTT & Broker MQTT

Por último, el componente MQTT es responsable de la comunicación remota de comandos y eventos entre el dispositivo y sistemas externos, como una aplicación web o móvil. Para este caso, se utilizó el ejemplo de la `mqtt5` de la esp. Del cual se obtuvo el procedimiento de, conectarse al broker, suscribirse a un tópico y recibir y publicar mensajes.

Configuración y Conexión:

La configuración del broker MQTT (URI y tópico) se almacena de forma persistente en la NVS. Esta configuración es ingresada inicialmente a través del servidor web en modo AP, junto con los datos de WiFi, y luego recuperada automáticamente al iniciar el sistema.

Al arrancar, el dispositivo:

- Recupera la configuración MQTT desde la NVS.
- Se conecta al broker público de HiveMQ utilizando la librería MQTT v5 de ESP-IDF.
- Se suscribe al tópico configurado para recibir comandos.

La función `mqtt_lib_init()` se encarga de inicializar el cliente MQTT, registrar el handler de eventos y establecer la conexión con el broker.

Recepción y Procesamiento de Comandos:

Cuando el dispositivo recibe un mensaje MQTT en el tópico suscrito, la librería ejecuta la función de callback `my_callback`. Esta función procesa el mensaje recibido y lo compara con los comandos válidos del sistema. Si el mensaje coincide con un comando reconocido, se llama a la función correspondiente del controlador de audio llamada `audio_controller_send_event`, que ejecuta la acción solicitada. Además, se actualiza el estado del LED para reflejar la acción.

Envío de Mensajes:

El componente MQTT también permite publicar eventos o logs al broker, utilizando la función `mqtt_lib_publish()`. Esto facilita la integración con otros sistemas que requieran monitorear el estado o el historial de eventos del dispositivo.

Integración con el Sistema:

El componente MQTT está integrado con el resto del proyecto, de forma que, recupera automáticamente los parámetros de conexión desde la NVS, permite controlar la reproducción y el volumen de forma remota mediante comandos MQTT, envía el historial de eventos al broker cuando se establece la conexión y actualiza el estado del LED según el comando recibido.

HEARTBEAT LED

En el caso del opcional del heartbeat del led, solo se agregaron las librerías de `led_strip` y `led`, las cuales cumplían la función de hacer parpadear el led. El único cambio que se realizó fue la utilización del `delay` de freertos ya que el utilizado anteriormente era bloqueante. Fue necesario también crear una máquina de estados para que el led se mantenga parpadeando mientras la música está sonando y que al pausarla se apague.

Máquina de estado para manejo del led:

La máquina de estado se cambia mediante la función `led_set_state`, que:

1. Envía el nuevo estado a la cola `g_led_state_queue`
2. La tarea principal `led_controller_task` procesa los cambios del estado y ejecuta las acciones correspondientes.

```
// Estados del LED para la máquina de estados
typedef enum {
    LED_STATE_OFF = 0,           // LED apagado
    LED_STATE_PLAY,             // LED parpadeando azul (reproduciendo)
    LED_STATE_PAUSE,            // LED apagado (pausado)
    LED_STATE_NEXT,             // LED azul sólido (cambio de pista)
    LED_STATE_PREVIOUS,         // LED azul sólido (cambio de pista)
    LED_STATE_ERROR              // LED rojo parpadeando (error)
} led_state_t;
```

TOUCH PAD

Para la implementación del touch_pad se utilizó una librería previamente empleada en laboratorios anteriores. Esta librería ejecuta dos funciones, una encargada de la configuración e inicialización del touch_pad de la esp, la cual establece los pines, y calibra los umbrales de sensibilidad; y otra función, llamada tp_read que se encarga de leer de forma continua el estado de los sensores táctiles. La ejecución de esta última, se realiza dentro de la tarea tp_read_task.

Por otro lado, en la estructura en nuestro caso case, encargada de identificar qué botón fue presionado, se llama a la función de audio_controller_send_event, a la cual se le pasa como parámetro el evento detectado por el touch_pad, para lograr que el led también responda a los comandos recibidos, dentro del mismo case se llama a la función led_set_state, avisando el estado del led como parámetro.

```
{
    last_time[i] = now;
    esp_err_t ret;
    switch (button[i]) {
        case TOUCH_PAD_NUM1: // VOL_UP
            ESP_LOGI(TAG, "► TOUCH VOL_UP: enviando evento VOL_UP");
            ret = audio_controller_send_event(AUDIO_EVENT_VOLUME_UP);
            led_set_state(LED_STATE_PLAY);
            if (ret != ESP_OK) {
                ESP_LOGE(TAG, "Error al enviar evento VOL_UP desde touch: %s", esp_err_to_name(ret));
            }
            break;

        case TOUCH_PAD_NUM3: // VOL_DOWN
            ESP_LOGI(TAG, "► TOUCH VOL_DOWN: enviando evento VOL_DOWN");
            ret = audio_controller_send_event(AUDIO_EVENT_VOLUME_DOWN);
            led_set_state(LED_STATE_PLAY);
            if (ret != ESP_OK) {
                ESP_LOGE(TAG, "Error al enviar evento VOL_DOWN desde touch: %s", esp_err_to_name(ret));
            }
            break;

        case TOUCH_PAD_NUM2: // PLAY
            ESP_LOGI(TAG, "► TOUCH PLAY: enviando evento PLAY");
            ret = audio_controller_send_event(AUDIO_EVENT_PLAY);
            led_set_state(LED_STATE_PLAY);
            if (ret != ESP_OK) {
                ESP_LOGE(TAG, "Error al enviar evento PLAY desde touch: %s", esp_err_to_name(ret));
            }
            break;
    }
}
```

NTP

Como parte de la implementación del sistema de registro de eventos, se integró un mecanismo de sincronización horaria utilizando NTP (Network Time Protocol), con el objetivo de registrar los eventos con marcas de tiempo reales y coherentes, incluso después de reinicios del dispositivo.

Implementación realizada

- Se utilizó la función `time(&now)` para obtener la hora actual desde el sistema, previamente sincronizada mediante NTP.
- La sincronización se configura automáticamente mediante `sntp` al utilizar `example_connect()`, una función de ejemplo proporcionada por ESP-IDF que gestiona la conexión Wi-Fi y activa el cliente NTP.
- La hora NTP queda disponible luego de una conexión exitosa a Internet, lo cual permite obtener un timestamp correcto para cada evento registrado.

Funcion para inicializar el NTP:

```
void ntp_initialize(void) {  
    ESP_LOGI(TAG, "Iniciando SNTP...");  
    sntp_setoperatingmode(SNTP_OPMODE_POLL); // CONSULTA LA HORA PERIODICAMENTE con el  
modo POLL  
    sntp_setservername(0, "pool.ntp.org"); // Le indicamos al servidor publico  
pool.ntp.org, 0 índice del servidor  
    sntp_init();  
}
```

Función de sincronización del NTP:

```
void ntp_wait_for_sync(void) {  
    time_t now = 0;  
    struct tm timeinfo = { 0 };  
  
    int retry = 0;  
    const int retry_count = 10;  
  
    ESP_LOGI(TAG, "Esperando sincronización NTP...");  
  
    while (timeinfo.tm_year < (2016 - 1900) && ++retry < retry_count) {  
        ESP_LOGI(TAG, "Intento de sincronización NTP... (%d/%d)", retry, retry_count);  
        vTaskDelay(pdMS_TO_TICKS(2000));  
        time(&now);  
        localtime_r(&now, &timeinfo);  
  
        // Log del tiempo actual para debug  
        ESP_LOGI(TAG, "Tiempo actual: %lld segundos desde epoch, año: %d",  
            (long long)now, timeinfo.tm_year + 1900);  
    }  
  
    if (timeinfo.tm_year >= (2016 - 1900)) {  
        // Configurar zona horaria de Uruguay (UTC-3)  
        // En el formato TZ, el signo es invertido: UTC-3 se escribe como "UYT3"  
        setenv("TZ", "UYT3", 1); // Uruguay Time (UTC-3)  
        tzset(); // Aplica la zona horaria al sistema  
  
        // Actualizar tiempo después de configurar zona horaria  
        time(&now);  
        localtime_r(&now, &timeinfo);  
  
        ESP_LOGI(TAG, "Hora sincronizada exitosamente!");  
  
        // Mostrar hora UTC  
        struct tm utc_time;  
        gmtime_r(&now, &utc_time);  
        ESP_LOGI(TAG, "Hora UTC: %04d-%02d-%02d %02d:%02d:%02d",  
            utc_time.tm_year + 1900, utc_time.tm_mon + 1, utc_time.tm_mday,  
            utc_time.tm_hour, utc_time.tm_min, utc_time.tm_sec);  
  
        // Mostrar hora local (Uruguay)  
        ESP_LOGI(TAG, "Hora local Uruguay: %04d-%02d-%02d %02d:%02d:%02d",  
            timeinfo.tm_year + 1900, timeinfo.tm_mon + 1, timeinfo.tm_mday,  
            timeinfo.tm_hour, timeinfo.tm_min, timeinfo.tm_sec);  
  
        ESP_LOGI(TAG, "Timestamp Unix: %lld", (long long)now);  
        ESP_LOGI(TAG, "Zona horaria: Uruguay (UTC-3)");  
        ESP_LOGI(TAG, "Verificación: UTC timestamp vs local time diferencia esperada: 3 horas");  
    } else {  
        ESP_LOGW(TAG, "No se logro sincronizar la hora via NTP");  
        ESP_LOGW(TAG, "Los timestamps del logger usaran el tiempo del sistema");  
  
        // Log del tiempo actual aunque no esté sincronizado  
        time(&now);  
        ESP_LOGW(TAG, "Tiempo no sincronizado: %lld segundos", (long long)now);  
    }  
}
```

Uso en el código:

En la función **logger_log_event**, se invocó `time(&now)` para capturar el tiempo actual en segundos desde Epoch, y se utilizó este valor como marca de tiempo (`evento.timestamp = now`) al registrar cada evento.

Se verificó que el sistema recupere correctamente la hora al inicio y que los eventos posteriores conserven una secuencia temporal realista.

```
esp_err_t logger_log_event(logger_event_type_t event_type)
{
    time_t now;
    time(&now);

    if (!g_logger_initialized) {
        ESP_LOGE(TAG, "Logger not initialized");
        return ESP_ERR_INVALID_STATE;
    }

    if (event_type < LOGGER_EVENT_PLAY || event_type > LOGGER_EVENT_STOP) {
        ESP_LOGE(TAG, "Invalid event type: %d", event_type);
        return ESP_ERR_INVALID_ARG;
    }

    esp_err_t err = logger_ring_buffer_add_event(event_type);
    if (err != ESP_OK) {
        ESP_LOGE(TAG, "Failed to add event to ring buffer: %s", esp_err_to_name(err));
        return err;
    }

    logger_request_save_async(); // petición asincrónica de guardado

    return ESP_OK;
}
```

La prueba se realizó ejecutando el proyecto en una placa ESP32-S2 física, conectada a una red Wi-Fi, verificando mediante logs (`ESP_LOGI`) que el timestamp recuperado coincidiera con la hora real. La incorporación de NTP asegura que todos los eventos registrados tengan una marca temporal confiable, lo que permite el correcto guardado de los logs de manera temporal en el **SPIFFS** del sistema.